

KITs College of Engineering Kolhapur (Empowered Autonomous)
Department of Computer Science and Engineering
OOP Experiments AY 2026-27

Lab Experiment No 1 - (First Week)

- Write a program to call function with different signature -
- Example functions signatures
 - Void fun()
 - Void fun(int)
 - Void fun(int*, int[])
- Student shall make use of Linux Programming Environment for writing the codes,
- The compiler should be “g++” compiler
- Give them exposure to memory allocation and management using the “malloc” function.
- Students should be able to use this memory to store different types of data.

Lab Experiment No 2 - (Second Week)

- Create an abstraction program with method abstraction
 - Declare header file with function signatures “add.h”
 - Declare function definition in separate file - “add.cpp”
 - Build “add.o” object file by compiling add.cpp program using command
 - g++ -c add.cpp
 - Develop a main method which calls the function add by including the header file “add.h” in the program and using the “add.o” file for compilation

Lab Experiment No 3 - (Third Week)

- Create an encapsulation program with class and method
 - Create a header file which defines class and declares methods in the class “math.h”
 - Write the method definitions of the declared method in new file “math.cpp” file
 - The “math.cpp” file shall import headerfile “math.h” as the class definition is in the headerfile
 - Build math.o file by following the process learned during the experiment no 2
 - Develop main method which calls the functions from the class

Lab Experiment No 4 - (Forth Week)

- Develop the “Stack.o” file and “Stack.h” file, the stack is integer datatype stack. (consider stack size as 5 or more)
- There should be no cin or cout statement in the stack class
- Develop a main method program which will make use of the stack.o file and call the function push, pop and peek to perform insertion, removal and display
- The stack overflow and underflow conditions shall be managed by returning a specific error code from push, pop and peek functions
- Example if push return -1 - its stack overflow, if push returns 0 - push is successful

Lab Experiment No 5 - (Fifth Week)

- Design a Student class containing student roll number, name, marks of subjects and appropriate member functions.
- Use constructors to initialize student information.
- Calculate total marks, percentage and grade and display the complete student result.

```
5  class Student
6  {
7  private:
8      int rollNo;
9      string name;
10     static const int SUBJECTS = 5;
11     float marks[SUBJECTS];
12     float total;
13     float percentage;
14     char grade;
15 public:
16     // Default Constructor
17     Student();
18     // Parameterized Constructor
19     Student(int r, string n, float m[]);
20     // Function to accept student information
21     void acceptDetails();
22     // Function to calculate total marks
23     void calculateTotal();
24     // Function to calculate percentage
25     void calculatePercentage();
26     // Function to determine grade
27     void calculateGrade();
28     // Function to display complete student result
29     void displayResult();
30 };
31
```

Experiment No 6 - (6th Week) (Copy Constructor)

Implement the structure given in the snapshot below , write your code by replacing the comment
“//write your code here block”

```
class Student{
public:
    int *rollno;
    // write your code here
};

int main(){
    Student s1(101);
    Student s2 = s1; // Calls the copy constructor

    cout<<"s1 rollno: "<<*s1.rollno<<endl;
    cout<<"s2 rollno: "<<*s2.rollno<<endl;

    Student *s3 = new Student(102);
    Student *s4 = new Student(*s3); // Calls the copy constructor

    // write your code here to change the value of rollno for s1 and s3 and print the
    // values of rollno for s1, s2, s3 and s4 to show that they are independent copies

    delete s3;
    delete s4;
    return 0;
}
```

Experiment No 7

Abstract class - implement the skeleton given below -

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double area() const = 0; // Pure virtual function

    void displayArea() const {
        cout << "Area: " << area() << '\n';
    }

    virtual ~Shape() = default;
};

class Circle : public Shape {
private:
    double radius;

public:
    //write your code here
};

class Rectangle : public Shape {
private:
    double length;
    double width;

public:
    //write your code here
};
```